Week 1 Lab Handout

API Development — Setting Up Your Team and Your Repo

Time: 110 minutes **Bring: your laptop, GitHub account, and the repo from your Web Dev app**

Before you start: what this semester actually is

You already built a working web app. This semester, that app becomes one node in a network — your API will be called by another team's app, and you'll call another team's API back. Everything you set up today (your team, your repo) is what makes that possible later, so don't rush it.

> **A note on today vs. next lab:** today you set up a plain GitHub repository so your team can start working together immediately. Next lab, your instructor will hand you a **template repository** that has an automated grading workflow already built in — you'll redo today's repo setup starting from that template, and add your instructor as a collaborator at that point too. Nothing you do today is wasted: your code just moves into the new repo next week. Today is about your team and your codebase, not about grading infrastructure yet.

Part A — Form your team (15 min)

1. Confirm your team of 3–4 (pre-assigned or self-formed per your instructor's instructions).
2. Decide whose Web Dev app becomes **the team app**. If multiple people had solo apps, pick the strongest one — the others will join as contributors, which is good practice for joining a codebase you didn't start (a very real developer skill).
3. **Find your ring position.** Your instructor will publish a numbered ring for your section (Team 1 → Team 2 → Team 3 → … → back to Team 1). Write down your **downstream partner** (the team that will consume your API) and your **upstream partner** (the team whose API you will consume).
4. Assign initial roles (these rotate every 4 weeks, so don't overthink it): **API Lead** (endpoint/contract decisions), **Backend Dev(s)** (endpoints and database logic), **Integration/QA Lead** (testing, including partner teams' APIs later), **Docs/DevOps Lead** (README, environment setup, deployment).

Part B — Audit your existing app (20 min)

Before you can expose an API, you need to know what's actually in your app.

As a team, list:

- Every "thing" your app stores (these will become your API's *resources* — e.g., tasks, users, posts, orders)
- Every action a user can take on those things (create, view, edit, delete, mark complete, search, filter, etc.)

Don't filter yet — write down everything. You'll narrow this to what's actually useful to your ring partner in Week 2.

Part C — Set up your team repository (30 min)

Today you're creating a **plain GitHub repository** — not a template, not connected to any grading system yet. That comes next lab. For now, the goal is simple: get your whole team working out of one shared codebase.

Step 1: Create a new repository

1. Go to github.com and sign in (create an account first if you don't have one).
2. Click the **+** in the top-right corner → **New repository**.

3. Name it something clear and consistent, e.g. `<yourteamname>-webapp` — you'll thank yourself later when you have 20 repos open across the semester.
4. Choose **Private** (recommended) or **Public** — either is fine for today.
5. **Do not** check "Add a README" — leave it empty, since you're about to push existing code into it.
6. Click **Create repository**.

Step 2: Import your existing web app's code

If your Web Dev app already lives in a local folder on your laptop:

```
cd path/to/your/existing/webdev/app
git init # skip this line if it's already a git repo
git add .
git commit -m "Import existing Web Dev app"
git branch -M main
git remote add origin https://github.com/<your-username>/<yourteamname>-webapp.git
git push -u origin main
```

If your app already lives in a *different* GitHub repo under someone's personal account, ask your instructor whether to push it here as shown above, or transfer that repo directly — either works, just pick one so the whole team is working from the same place.

Confirm it worked: refresh the repo page on GitHub.com — you should see your actual project files listed, not an empty repo.

Step 3: Add every teammate as a collaborator

1. In your new repo, go to **Settings** → **Collaborators**.
2. Click **Add people** and add every teammate by their GitHub username or email.
3. Have each teammate accept the invite (check their email or GitHub notifications).

Do not add your instructor yet — that happens next lab, once you've moved to the template repository.

Step 4: Confirm access actually works

Pick a teammate who is *not* the person who created the repo. Have them clone the repo, make a trivial change (e.g. create `CONTRIBUTORS.md` and add everyone's name), then commit and push it directly to `main`:

```
git clone https://github.com/<your-username>/<yourteamname>-webapp.git
git add CONTRIBUTORS.md
git commit -m "Add contributors list"
git push
```

If that push succeeds, everyone's access is confirmed working — a much better time to discover a permissions problem than during Week 4 when a real deadline is on the line.

Part D — Preview: How Weekly Grading Will Work (10 min)

No hands-on work here today — just so you know what's coming next lab and aren't caught off guard.

Starting next lab, once you have the template repository:

• Every week, you'll submit your work on a branch named `weekNN-submission` (e.g. `week05-submission`), and open a Pull Request from it.

• An automated workflow will check your PR within a couple of minutes and post a comment showing what passed and what didn't — this is part of your grade for that week, automatically.

• Some things will always be graded by your instructor directly (design decisions, written reflections, live demos) — the automation only covers what's genuinely checkable by a machine.

You don't need to do anything with this yet. Just know that the plain repo you set up today is the rehearsal — next lab's repo is the one that actually counts.

Part E — Write your Team Charter (30 min)

Create `TEAM_CHARTER.md` at your repo root containing:

1. Team name and all members with their starting role
2. One-paragraph summary of your existing app (what it does, who it's for)
3. Your Part B audit — every resource and action you listed
4. Your ring position — who you'll consume from, who will consume you

Commit and push this file directly to `main`:

```
git add TEAM_CHARTER.md
git commit -m "Add team charter"
git push
```

(There's no branch-naming convention to follow yet — that starts next lab once your work is happening in the template repo.)

This week's deliverable checklist

- [ ] Team formed, roles assigned, ring position confirmed
- [ ] GitHub repository created, existing app code imported and pushed
- [ ] All teammates added as collaborators, access confirmed by a non-owner push
- [ ] TEAM_CHARTER.md written, complete, and pushed to main

Common problems and quick fixes

Problem	Fix
git push rejected with a “permission denied” error	The collaborator invite probably hasn't been accepted yet — check the invited teammate's email/GitHub notifications
git push rejected with “repository not found”	Double-check the remote URL matches your actual repo — a typo in the username or repo name is the usual cause
Not sure who your ring partners are	Check the ring diagram posted in your section's course space — ask your instructor if it's unclear, don't guess
Existing app was never in git / no local git history	Run git init inside the project folder first, then follow Step 2 as normal — you'll just have one big “initial import” commit instead of your original history, which is fine
Teammate can see the repo but can't push	Confirm they were added as a Collaborator (Settings → Collaborators), not just given the link — a link alone doesn't grant write access on a private repo

Next lab: your instructor hands you the template repository. You'll repeat this same setup — new repo, code imported, teammates added — this time from the template, with your instructor added as a collaborator too. That's the repo you'll actually submit work from for the rest of the semester.